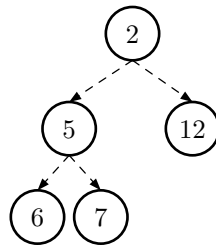


1 Heaperations

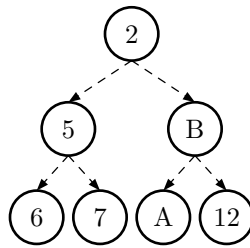
Consider the following min-heap:



(a) Draw the heap after inserting 4, 13, and 3 in that order.

(b) Now, draw the heap after 2 calls of **removeMin**.

- (c) Suppose we recover the original heap and insert two elements of unknown value, A and B , **in that order**.



For each of the following conditional statements, write whether they are **always** true, **sometimes** true, or **never** true, and briefly explain why.

$A < B$

$B > 5$

$A \geq 12$

- (d) Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are distinct.

removeMin has a best-case runtime of _____ and a worst-case runtime of _____

insert has a best-case runtime of _____ and a worst-case runtime of _____

A _____ or _____ traversal on a min-heap may output the elements in sorted order. (Assume there are at least 3 elements in the min-heap)

The fourth smallest element in a min-heap with 1000 distinct elements can appear in _____ different places in the heap.

Given a min-heap with 2^{N-1} **distinct** elements, for an element

- to be on the second level it must be less than _____ element(s) and greater than _____ element(s).
- to be on the bottommost level it must be less than _____ element(s) and greater than _____ element(s).

Hint: A complete binary tree (with a full last-level) has $2^N - 1$ elements, with N being the number of levels.

2 Heaps, the Hard Way

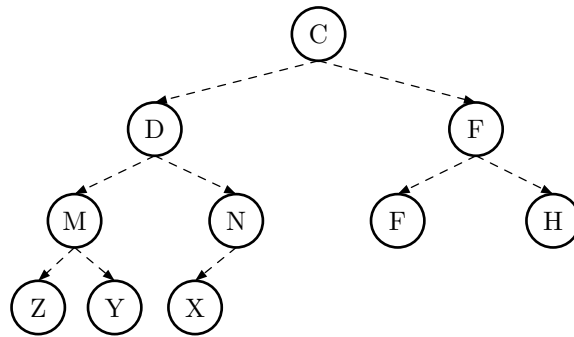
In lecture, we implemented a heap using a backing array. We could also have done so with a tree representation, given below:

```
private class HeapNode {
    private T item;
    private HeapNode left;
    private HeapNode right;

    HeapNode(T i) {
        item = i;
    }
}
```

One issue with this approach is that it is somewhat tricky to find where to insert new nodes. In this problem, we'll walk through this in some detail.

- (a) Suppose we have a heap that currently looks like the figure below, where the integer beside each node is the “node number”, i.e. position in the heap, of each item.



If we add a new item, which node will be the parent of the new node (before doing any **bubbleUp** operations)? Why?

- (b) Fill in the `isRightChild(int i)` and `parent(int i)` methods. Assume for `isRightChild` that `i` is nonnegative and for `parent` that `i` is positive. Notice that the root has node number zero.

- `isRightChild` returns true if the node number `i` corresponds to a right child. E.g. `isRightChild(2)` and `isRightChild(4)` should evaluate to `true`, and `isRightChild(0)` and `isRightChild(1)` should evaluate to `false`
- `parent` returns the node number of the parent of the given node `i`. E.g. `parent(9)` should evaluate to 4

```
private boolean isRightChild(int i) {
    return _____;
}

private int parent(int i) {
    return _____;
}
```

- (c) Fill in the `followPath(List path)` method so that it matches the given javadoc comment. Assume the given path is valid.

```
/** Return the HeapNode at the end of the given path.
 * For example if path is: false, true, false
 * We will go left, right, left, landing at node #9 */
```

```
private HeapNode followPath(List<Boolean> path) {
    HeapNode p = root;

    for (_____ : _____){
        if (_____) {
            _____;
        } else {
            _____;
        }
    }
    return p;
}
```

- (d) The method below will return the i^{th} node. For example, if we call **getNode(9)** on the example heap, we will get the HeapNode containing X.

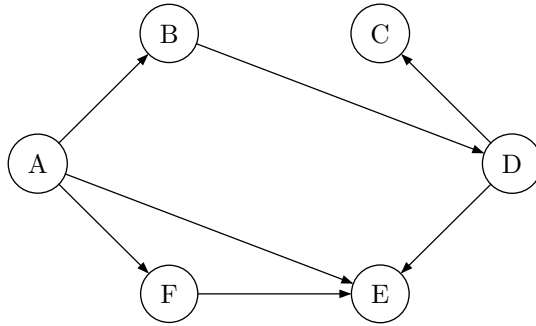
```
private HeapNode getNodeI(int i) {
    List<Boolean> pathToI = constructPath(i);
    HeapNode nodeNumberI = followPath(pathToI);
    return nodeNumberI;
}
```

Fill in the constructPath method so that the getNodeI method above works correctly. You may use any method from the previous parts of this problem. *Hint: Recall that LinkedList has methods addFirst, addLast, removeFirst, and removeLast.*

```
private List<Boolean> constructPath(int i) {
    LinkedList<Boolean> list = new LinkedList<>();

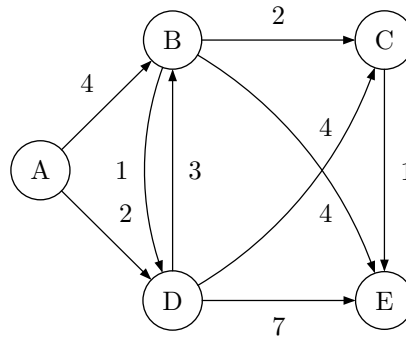
    while (_____) {
        _____;
        _____;
    }
    return list;
}
```

3 Graphs



- (a) Represent the above graph as an adjacency list and adjacency matrix
- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.

4 Dijkstra's Algorithm



- (a) Given the above graph, run Dijkstra's algorithm starting at node A. For each iteration, write down the entire state of the algorithm. This includes the value **dist(v)** for all vertices **v** as well as what node was popped off of the fringe for that iteration.

*Note: If you want to keep track of the vertices traversed along the shortest paths from A to every other node in the graph, you will need to maintain an **edgeTo** array.*

- (b) What must be true about our graph in order to guarantee Dijkstra's will return the shortest path's tree to every vertex? Draw an example of a graph that demonstrates why Dijkstra's might fail if we do not satisfy this condition.

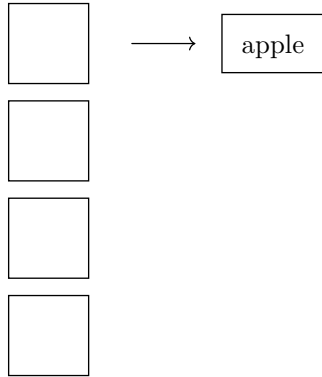
5 Hashing Practice

Given the provided `hashCode` implementation, hash the items listed below with separate chaining (the first item is already inserted for you). Assume the load factor is 1. Use geometric resizing with a resize factor of 2.

You may draw more boxes to extend the array when you need to resize.

```
/** Returns 0 if word begins with 'a', 1 if it begins with 'b', etc. */  
public int hashCode() {  
    return (int)(word.charAt(0) - 'a');  
}
```

```
["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]
```



6 LLRB-Chaining

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

- (a) Using this hash table,

The runtime of **contains** is, in the worst case:

Runtime: Θ (_____)

The runtime of **contains** is, in the best case:

Runtime: Θ (_____)

- (b) Evaluate the following statements as **true** or **false**, and briefly explain why.

“One advantage of using an LLRB for buckets is that it becomes possible to efficiently iterate over the keys in ascending order”

“Assuming items in the hash table have good spread, we expect that an LLRB bucket would yield significantly better performance for **contains** and **add** than using an **ArrayList**”

7 Spread and Validity

Suppose we have the Dog class below.

```
public class Dog {
    private static int dogsCreated = 0;
    private int age;
    private final int microchipID;
    public String name;
    public String owner;

    public Dog(int age, String name, String owner) {
        Dog.dogsCreated += 1;
        this.age = age;
        this.name = name;
        this.owner = owner;
        this.microchipID = dogsCreated;
    }

    public void birthday() {
        this.age += 1;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Dog)) {
            return false;
        }
        Dog other = (Dog) o;
        return this.name.equals(other.name) && this.microchipID == other.microchipID;
    }

    @Override
    public int hashCode() {
        // Implementation omitted, see part (a)
    }

    ...
}
```

- (a) For each of the following implementations of the Dog's hashCode method, determine if it is **valid with good spread**, **valid with bad spread**, **valid with no spread**, or **invalid**.

Recall that “validity” depends on the following rules:

- Two objects that are equal (by the **equals** method) must have the same hash code
- An object's hash code only changes if its equality changes

```
@Override
public int hashCode() {
    return this.age;
}
```

```
@Override
public int hashCode() {
    return this.microchipID;
}
```

```
@Override
public int hashCode() {
    return (int) this.name.charAt(0);
}
```

```
@Override
public int hashCode() {
    return this.owner.hashCode();
}
```

- (b) If a HashMap contains two or more items with equal keys, how might have this happened? Give one reason. Assume that we are referring to the built in java HashMap. Note that this part is independent from the previous; i.e. the keys aren't necessarily Dogs.